现代 C++ 处理异常和错误的最佳做法

2024/03/28

在新式 C++ 中，在大多数情况下，报告和处理逻辑错误与运行时错误的首选方式是使用异常。 当堆栈可能在检测错误的函数与具有错误处理上下文的函数之间包含多个函数调用时，这种方式尤其有用。 异常为检测错误的代码提供正式的、妥善定义的方式，以将信息向上传递到调用堆栈。

对异常代码使用异常

程序错误通常分为两类：

- 编程错误导致的逻辑错误。 例如，“索引超出范围”错误。
- 超出程序员控制的运行时错误。 例如，“网络服务不可用”错误。

在 C 样式的编程和 COM 中，错误报告的管理方式是返回一个表示错误代码或特定函数的状态代码的值，或者设置一个全局变量，调用方可以在每次执行函数调用后选择性地检索该变量来查看是否报告了错误。 例如，COM 编程使用 `HRESULT` 返回值将错误传达给调用方。 Win32 API 提供 `GetLastError` 函数来检索调用堆栈报告的最后一个错误。 在这两种情况下，都需要由调用方识别代码并相应地做出响应。 如果调用方未显式处理错误代码，则程序可能会在不发出警告的情况下崩溃。 或者，它可能会继续使用错误的数据执行，并生成错误的结果。

新式 C++ 中优先使用异常的原因如下：

- 异常会强制调用代码识别并处理错误状态。 未经处理的异常会停止程序执行。
- 异常跳转到调用堆栈中可以处理错误的位置。 中间函数可以让异常传播。 这些函数不必与其他层协调。
- 引发异常后，异常堆栈展开机制将根据妥善定义的规则销毁范围内的所有对象。
- 异常可以在检测错误的代码与处理错误的代码之间实现明确的分离。

以下简化示例演示了 C++ 中引发和捕获异常的必要语法：

C++

```
#include <stdexcept>
#include <limits>
#include <iostream>

using namespace std;

void MyFunc(int c)
{
    if (c > numeric_limits< char> ::max())
    {
        throw invalid_argument("MyFunc argument too large.");
    }
    //...
}

int main()
{
    try
    {
        MyFunc(256); //cause an exception to throw
    }

    catch (invalid_argument& e)
    {
        cerr << e.what() << endl;
        return -1;
    }
    //...
    return 0;
}
```

C++ 中的异常类似于 C# 和 Java 等语言中的异常。 在 `try` 块中，如果引发某个异常，类型与该异常的类型匹配的第一个关联 块将捕获该异常 `catch`。 换言之，执行将从 `throw` 语句跳转到 `catch` 语句。 如果未找到可用的 catch 块，则调用 `std::terminate` 并且程序会退出。 在 C++

中可以引发任何类型；但是，我们建议引发直接或间接派生自 `std::exception` 的类型。 在前面的示例中，异常类型 invalid_argument 是在标准库的 <stdexcept> 头文件中定义的。 C++ 既不提供也不需要 **`finally`** 块来确保在引发异常时释放所有资源。 资源采集是使用智能指针的初始化 (RAII) 习语，它提供所需的功能来清理资源。 有关详细信息，请参阅如何：针对异常安全性进行设计。 有关 C++ 堆栈展开（回退）机制的信息，请参阅异常和堆栈展开。

基本准则

在任何编程语言中实现可靠的错误处理都颇有挑战性。 尽管异常提供多项功能来支持妥善的错误处理，但它们不能代你解决一切问题。 为了实现异常机制的优势，请在设计代码时考虑到异常。

- 使用断言检查应始终为 true 或始终为 false 的条件。 使用异常来检查可能发生的错误，例如公共函数参数的输入验证错误。 有关详细信息，请参阅异常与断言部分。
- 当处理错误的代码与通过一个或多个中间函数调用检测错误的代码分离时，请使用异常。 当处理错误的代码与检测错误的代码紧密耦合时，请考虑是否在性能关键型循环中使用错误代码。
- 对于每个可能引发或传播异常的函数，请提供三项异常保证之一：强保证、基本保证或 nothrow（`noexcept`）保证。 有关详细信息，请参阅如何：针对异常安全性进行设计。
- 通过值引发异常，通过引用捕获异常。 不要捕获无法处理的异常。
- 不要使用 C++11 中已弃用的异常规范。 有关详细信息，请参阅异常规范和 noexcept 部分。
- 使用适用的标准库异常类型。 从 exception 类层次结构派生自定义的异常类型。
- 不要允许异常从析构函数或内存解除分配函数中逃逸。

异常和性能

如果未引发异常，则异常机制的性能开销极低。 如果引发异常，则堆栈遍历和展开的开销与函数调用的开销大致相当。 进入 **`try`** 块后，需要使用其他数据结构来跟踪调用堆栈，如果引发异常，则还需要使用更多指令来展开堆栈。 但是，在大多数情况下，性能和内存占用的开销并不高。 异常对性能的不利影响可能仅在内存受限的系统上才比较明显。 或者，这种影响在性能关键型循环中（其中的错误可能经常发生，并且

处理错误的代码与报告错误的代码之间存在紧密耦合）可能比较明显。 无论在哪种情况下，不进行分析和测量就不可能知道异常的实际开销。 即使在开销很高的极少数情况下，也可将这种开销与设计良好的异常策略所提供的更高正确性、更方便的维护性和其他优势进行权衡。

异常与断言

异常和断言是用于检测程序中运行时错误的两种不同机制。 如果所有代码都正确，可以使用 `assert` 语句来测试开发过程中始终应为 true 或 false 的条件。 使用异常来处理此类错误是没有意义的，因为错误指示的是代码中必须修复的问题。 它并不表示程序在运行时必须从中恢复的状态。 `assert` 在语句中停止执行，以便可以在调试器中检查程序状态。 异常从第一个适当的 catch 处理程序继续执行。 即使代码正确，也可以使用异常来检查在运行时可能发生的错误状态，例如“找不到文件”或“内存不足”。 异常可以处理这些状态，即使恢复只是将消息输出到日志并结束程序。 始终使用异常来检查公共函数的参数。 即使函数没有错误，也可能无法完全控制用户传递给它的参数。

C++ 异常与 Windows SEH 异常

C 和 C++ 程序都可以使用 Windows 操作系统中的结构化异常处理 (SEH) 机制。 SEH 中的概念类似于 C++ 异常中的概念，只不过 SEH 使用 **`__try`**、**`__except`** 和 **`__finally`** 构造，而不是 **`try`** 和 **`catch`**。 在 Microsoft C++ 编译器 (MSVC) 中，为 SEH 实现了 C++ 异常。 但是，在编写 C++ 代码时，请使用 C++ 异常语法。

有关 SEH 的详细信息，请参阅结构化异常处理 (C/C++)。

异常规范和 `noexcept`

C++ 中引入了异常规范，作为一种用于指定函数可能引发的异常的方式。 但经证实，异常规范会在实践中造成问题，并且在 C++11 草案标准中已遭弃用。 我们建议不要使用除 **`throw`**（表示函数不允许异常逃逸）以外的 `throw()` 异常规范。 如果必须使用已弃用 `throw( type-name )` 形式的异常规范，则 MSVC 支持会受限制。 有关详细信息，请参阅异常规范 (throw)。 C++11 中引入了 **`noexcept`** 说明符作为 `throw()` 的首选替代项。

另请参阅

如何：异常和非异常代码之间的接口

C++ 语言参考

C++ 标准库